

Rapport Technique Complet

Type de projet : Embarqué / Robotique / Vision par Ordinateur

Matériels : Arduino Uno, Raspberry Pi, L298N, Webcam

Langages : Python 3, C++ (Arduino), protocole MQTT

Bibliothèques : OpenCV, MediaPipe, Paho-MQTT, PySerial

Table des matières

1	Introduction et Présentation du Projet	3
1.1	Contexte et Motivation	3
1.2	Architecture Générale du Système	3
1.3	Gestes Reconnus	4
2	Matériel Nécessaire	5
2.1	Liste du Matériel	5
2.2	Schéma de Câblage Arduino – L298N – Moteurs	6
3	Installation des Logiciels et Dépendances	7
3.1	Environnement côté Ordinateur / Raspberry Pi	7
3.1.1	Python 3 et pip	7
3.1.2	Création d'un environnement virtuel (recommandé)	7
3.1.3	Bibliothèques Python requises	7
3.2	Installation du Broker MQTT (Mosquitto)	8
3.3	Environnement côté Arduino	8
3.3.1	Identifier le port série sous Linux	9
4	Code Source Commenté	10
4.1	Sketch Arduino : Contrôle des Moteurs (<code>motors.ino</code>)	10
4.2	Script Python : Pont MQTT – Série (<code>bridge.py</code>)	11
4.3	Script Python : Détection de Gestes (<code>vision.py</code>)	12
4.3.1	Configuration et paramètres	12
4.3.2	Détection de l'état des doigts	13
4.3.3	Boucle principale de capture et d'envoi	15
5	Flux de Données et Protocoles	18
5.1	Le Protocole MQTT	18
5.2	Communication Série (UART)	18
5.3	Diagramme de Séquence Complet	18
6	Mise en Œuvre et Démarrage	20
6.1	Ordre de Démarrage	20
6.2	Résolution des Problèmes Courants	21

7	Pistes d'Amélioration	22
7.1	Améliorations Fonctionnelles	22
7.2	Améliorations Techniques	22
8	Conclusion	23

Chapitre 1

Introduction et Présentation du Projet

1.1 Contexte et Motivation

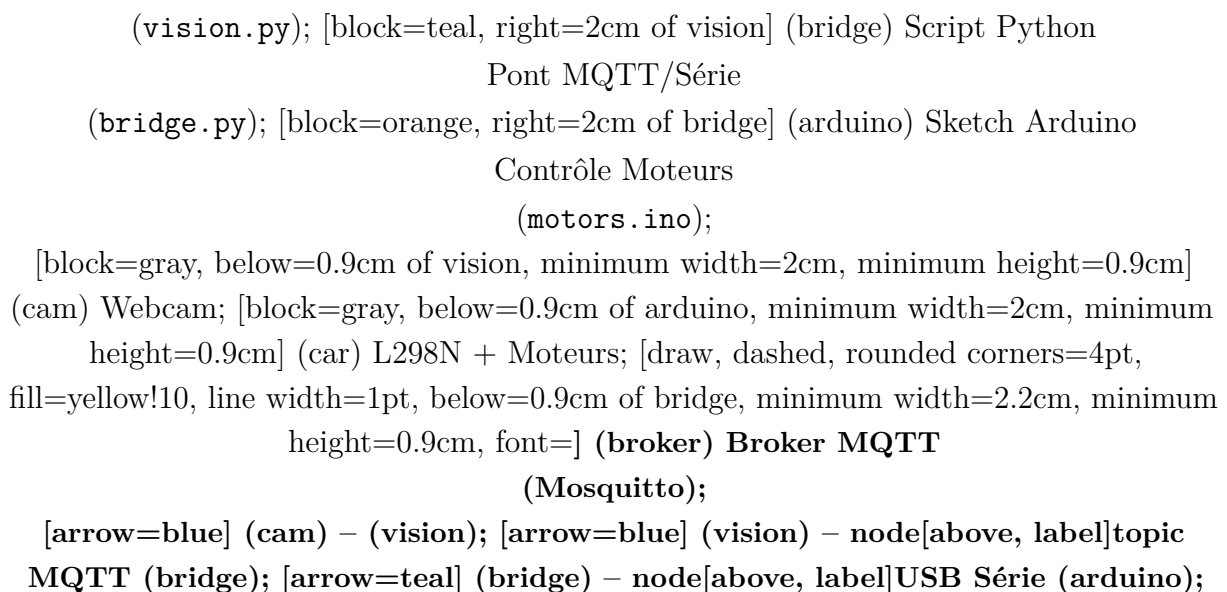
Ce projet propose la réalisation d'une **voiture télécommandée sans contact**, pilotée uniquement par des gestes de la main captés via une webcam. L'objectif est de combiner plusieurs domaines de l'informatique embarquée et de l'intelligence artificielle :

- la **vision par ordinateur** (détection de la main en temps réel) ;
- la **communication réseau légère** (protocole MQTT) ;
- la **programmation embarquée** (contrôle de moteurs avec Arduino).

Le résultat est un système complet, de bout en bout, où un geste de la main se traduit en mouvement physique d'un véhicule en quelques dizaines de millisecondes.

1.2 Architecture Générale du Système

Le projet repose sur **trois composants logiciels** distincts, qui communiquent entre eux selon le schéma suivant :



```
[arrow=orange] (arduino) - (car); [arrow=gray, dashed] (vision.south) - ++(0,-0.35)
-| (broker); [arrow=gray, dashed] (broker) - (bridge.south);
```

i Rôle de chaque composant

vision.py Capture la vidéo, détecte la main, classifie le geste, publie la commande sur le broker MQTT.

bridge.py Souscrit au topic MQTT, reçoit la commande et la transmet à l'Arduino via le port série USB.

motors.ino Lit la commande série et pilote le pont-H L298N qui alimente les moteurs.

1.3 Gestes Reconnus

Le tableau ci-dessous résume la correspondance geste → commande → mouvement.

Émoji	Geste	Commande MQTT	Mouvement
	Poing fermé	stop	Arrêt complet
	5 doigts ouverts	avant	Avancer
	Index seul	gauche	Tourner à gauche
	Index + Majeur	droite	Tourner à droite

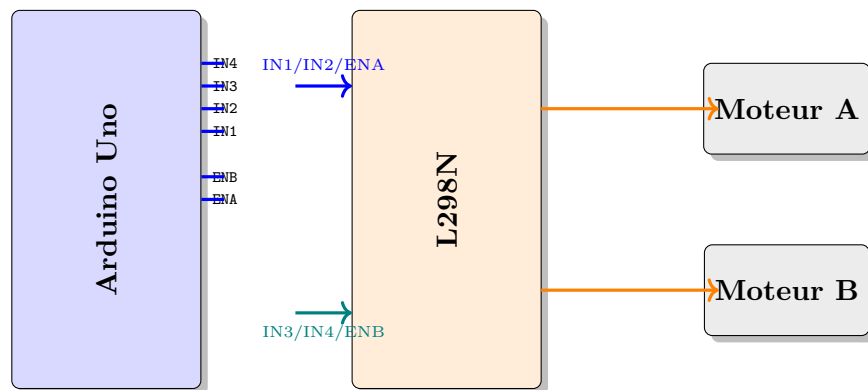
Chapitre 2

Matériel Nécessaire

2.1 Liste du Matériel

Composant	Modèle type	Rôle	Quantité
Microcontrôleur	Arduino Uno R3	Pilote les moteurs via L298N	1
Ordinateur/SBC	Raspberry Pi 4	Exécute les scripts Python	1
	(ou PC Linux/-Windows)		
Pont-H moteur	L298N	Interface entre Arduino et moteurs DC	1
Moteurs DC	Moteurs 3-6V TT	Entraînent les roues	2 ou 4
Châssis voiture	Kit 2WD/4WD	Structure mécanique	1
Webcam	USB 640×480 min	Capture la vidéo de la main	1
Câble USB	USB-A vers USB-B	Communication série Raspberry Pi ↔ Arduino	1
Alimentation	Batterie 7.4V LiPo ou 6×AA	Alimentation moteurs et carte	1
Câbles Dupont	Mâle/Femelle	Connexions entre composants	~20

2.2 Schéma de Câblage Arduino – L298N – Moteurs



Correspondance des broches

Pin Arduino	Broche L298N	Fonction
8	IN1	Direction moteur A
9	IN2	Direction moteur A
5	ENA	Vitesse moteur A (PWM)
10	IN3	Direction moteur B
11	IN4	Direction moteur B
6	ENB	Vitesse moteur B (PWM)
GND	GND	Masse commune

Important : Les pins ENA et ENB du L298N doivent être *débrochés* (jumper retiré) pour être contrôlables par PWM depuis l'Arduino.

Chapitre 3

Installation des Logiciels et Dépendances

3.1 Environnement côté Ordinateur / Raspberry Pi

3.1.1 Python 3 et pip

Python 3.8 ou supérieur est requis. Sur Raspberry Pi OS ou Ubuntu :

```
1 sudo apt update && sudo apt upgrade -y
2 sudo apt install python3 python3-pip python3-venv -y
```

Listing 3.1 – Installation de Python et pip

3.1.2 Création d'un environnement virtuel (recommandé)

```
1 python3 -m venv env_voiture
2 source env_voiture/bin/activate    # Linux / macOS
3 # Sur Windows : env_voiture\Scripts\activate
```

Listing 3.2 – Environnement virtuel Python

3.1.3 Bibliothèques Python requises

```
1 pip install opencv-python mediapipe paho-mqtt pyserial
```

Listing 3.3 – Installation des dépendances Python

Paquet	Rôle	Version testée
opencv-python	Capture vidéo, affichage, traitement image	4.8+
mediapipe	Détection et suivi de la main (landmarks)	0.10+
paho-mqtt	Client MQTT (publication/souscription)	1.6+
pyserial	Communication série avec l'Arduino	3.5+

3.2 Installation du Broker MQTT (Mosquitto)

Le broker MQTT joue le rôle d'intermédiaire : il reçoit les messages du script de détection et les redistribue au script pont. Il doit être installé sur la même machine que `bridge.py` (ou accessible sur le réseau local).

```

1 # Sur Debian/Ubuntu/Raspberry Pi OS
2 sudo apt install mosquitto mosquitto-clients -y
3 sudo systemctl enable mosquitto
4 sudo systemctl start mosquitto
5
6 # Vérification
7 mosquitto_pub -h localhost -t test/commande -m "avant"
8 mosquitto_sub -h localhost -t test/commande

```

Listing 3.4 – Installation de Mosquitto

Configuration réseau

Si `vision.py` s'exécute sur une machine *différente* de `bridge.py`, remplacez "localhost" dans `vision.py` par l'adresse IP de la machine hébergeant Mosquitto (ex. "10.10.33.32" comme dans le code fourni). Assurez-vous que le port **1883** est ouvert dans le pare-feu.

3.3 Environnement côté Arduino

1. Télécharger et installer l'**Arduino IDE** (version 2.x recommandée) : <https://www.arduino.cc/en/software>
2. Brancher l'Arduino Uno via USB.
3. Sélectionner le bon port série (/dev/ttyACM0 sous Linux, COM3 sous Windows, etc.).
4. Aucune bibliothèque tierce n'est nécessaire pour le sketch moteur.

3.3.1 Identifier le port série sous Linux

```
1 ls /dev/ttyACM*    # Typique pour Arduino Uno
2 ls /dev/ttyUSB*    # Parfois pour clones CH340
3
4 # Ou plus pr c i s m e n t :
5 dmesg | grep tty | tail -5
```

Listing 3.5 – Trouver le port série de l’Arduino

Chapitre 4

Code Source Commenté

4.1 Sketch Arduino : Contrôle des Moteurs (motors.ino)

Ce code est le plus simple des trois : il lit une commande textuelle sur le port série et active le pont-H L298N en conséquence.

```
1 //      Pins moteurs

2 int IN1 = 8,  IN2 = 9,  ENA = 5;  // Moteur gauche (A)
3 int IN3 = 10, IN4 = 11, ENB = 6;  // Moteur droit  (B)
4
5 int VITESSE = 130;  // PWM : 0-255, ajustez selon votre voiture
6
7 void setup() {
8     Serial.begin(9600);
9     // D clarer toutes les pins en sortie
10    pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT); pinMode(ENA, OUTPUT);
11    pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
12    arreter();  // Etat initial s curis
13 }
14
15 void loop() {
16     if (Serial.available()) {
17         String cmd = Serial.readStringUntil('\n');
18         cmd.trim();  // Supprimer \r et espaces parasites
19
20         if      (cmd == "avant")  avancer();
21         else if (cmd == "stop")   arreter();
22         else if (cmd == "droite") tournerDroite();
23         else if (cmd == "gauche") tournerGauche();
24     }
25 }
26
27 //      Fonctions de mouvement

28 void avancer() {
29     analogWrite(ENA, VITESSE); analogWrite(ENB, VITESSE);
```

```

30     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);    // Gauche :
        marche avant
31     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);    // Droite :
        marche avant
32 }
33
34 void arreter() {
35     analogWrite(ENA, 0); analogWrite(ENB, 0);
36     digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
37     digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
38 }
39
40 void tournerDroite() {
41     analogWrite(ENA, VITESSE); analogWrite(ENB, VITESSE);
42     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);    // Gauche
        avance
43     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);    // Droite
        recule
44 }
45
46 void tournerGauche() {
47     analogWrite(ENA, VITESSE); analogWrite(ENB, VITESSE);
48     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);    // Gauche
        recule
49     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);    // Droite
        avance
50 }

```

Listing 4.1 – Sketch Arduino complet – contrôle moteurs

💡 Ajustement de la vitesse

La variable VITESSE (valeur PWM entre 0 et 255) contrôle la puissance envoyée aux moteurs. Commencez avec 100–130 puis augmentez progressivement. Une valeur trop faible ne fera pas tourner les moteurs (couple insuffisant); une valeur trop élevée peut surchauffer le L298N.

4.2 Script Python : Pont MQTT – Série (bridge.py)

Ce script fait le lien entre le réseau (broker MQTT) et l'Arduino (port série USB). Il tourne en permanence sur la machine connectée à l'Arduino.

```

1 import paho.mqtt.client as mqtt
2 import serial
3 import time
4

```

```

5  #          Port s r i e vers Arduino

6  # V r i f i e z avec : ls /dev/ttyUSB* ou ls /dev/ttyACM*
7  ser = serial.Serial('/dev/ttyACM1', 9600, timeout=1)
8  time.sleep(2)    # Attendre l'initialisation de l'Arduino
9
10 BROKER = "localhost"
11 TOPIC  = "test/commande"
12
13 def on_message(client, userdata, msg):
14     """Callback d c l e n c h      chaque message re u sur le topic."""
15     commande = msg.payload.decode().strip()
16     print(f"Re u : {commande}")
17     ser.write((commande + "\n").encode())    # Envoi s r i e vers
        Arduino
18
19 client = mqtt.Client()
20 client.on_message = on_message
21 client.connect(BROKER, 1883)
22 client.subscribe(TOPIC)
23
24 print("En attente de commandes...")
25 client.loop_forever()    # Boucle bloquante MQTT

```

Listing 4.2 – Script Python – Pont MQTT/Série



Points d'attention

- Le port `/dev/ttyACM1` doit correspondre au port réel de votre Arduino (utilisez `ls /dev/ttyACM*` pour le trouver).
- Le `time.sleep(2)` est indispensable : l'Arduino redémarre lors de l'ouverture du port série et n'est pas prêt immédiatement.
- Le `BROKER = "localhost"` implique que Mosquitto tourne sur la même machine. Modifiez cette valeur si nécessaire.

4.3 Script Python : Détection de Gestes (vision.py)

C'est le cœur du projet. Il capture la vidéo, détecte la main via MediaPipe, classe le geste et publie la commande via MQTT.

4.3.1 Configuration et paramètres

```

1  import cv2

```

```

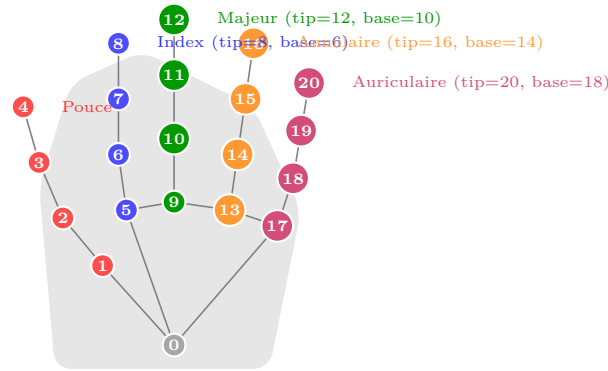
2 import mediapipe as mp
3 import paho.mqtt.client as mqtt
4
5 #           Configuration MQTT
6
7 BROKER = "10.10.33.32" # IP de la machine hbergeant Mosquitto
8 PORT   = 1883
9 TOPIC  = "test/commande"
10
11 client = mqtt.Client()
12 client.connect(BROKER, PORT)
13 client.loop_start() # Thread MQTT non-bloquant
14
15 #           Initialisation MediaPipe Hands
16
17 mp_hands = mp.solutions.hands
18 mp_drawing = mp.solutions.drawing_utils
19 hands = mp_hands.Hands(
20     static_image_mode=False, # Mode video (tracking continu)
21     max_num_hands=1, # Une seule main détectée
22     min_detection_confidence=0.85, # Seuil de détection initiale
23     min_tracking_confidence=0.85 # Seuil de suivi inter-frames
24 )
25
26 #           Seuils de reconnaissance
27
28 SEUIL_DOIGT_LEVE = 0.06 # Ecart min (y) pour considérer un
29     doigt levé
30 SEUIL_DOIGT_PLIE = 0.06 # Ecart max (y) pour le poing fermé
31 SEUIL_POUCE = 0.05 # Seuil horizontal pour le pouce
32 SEUIL_PAUME_OUVERTE = 4 # Nb minimum de doigts pour "avant"
33 NB_FRAMES_CONFIRM = 3 # Frames identiques pour confirmer un
34     geste

```

Listing 4.3 – Initialisation – vision.py

4.3.2 Détection de l'état des doigts

MediaPipe fournit 21 *landmarks* (points clés) par main. La figure suivante montre leur numérotation.



```

1 def etat_doigts(hand_landmarks):
2     """Retourne [pouce, index, majeur, annulaire, auriculaire] : True
3     = lev ."""
4     lm = hand_landmarks.landmark
5     doigts = []
6
7     # Pouce : comparaison horizontale (x) entre tip(4) et joint(3)
8     doigts.append(abs(lm[4].x - lm[3].x) > SEUIL_POUCE)
9
10    # 4 doigts : comparaison verticale (y) tip vs base
11    # En image, y augmente vers le bas => lev si base.y > tip.y
12    tips = [8, 12, 16, 20]
13    bases = [6, 10, 14, 18]
14    for tip, base in zip(tips, bases):
15        ecart = lm[base].y - lm[tip].y
16        doigts.append(ecart > SEUIL_DOIGT_LEVE)
17
18    return doigts # [pouce, index, majeur, annulaire, auriculaire]
19
20 def est_poing_strict(hand_landmarks):
21     """Retourne True si TOUS les doigts sont clairement repliés."""
22     lm = hand_landmarks.landmark
23     tips = [8, 12, 16, 20]
24     bases = [6, 10, 14, 18]
25     for tip, base in zip(tips, bases):
26         if lm[base].y - lm[tip].y > SEUIL_DOIGT_PLIE:
27             return False # Un doigt encore lev
28     if abs(lm[4].x - lm[3].x) > SEUIL_POUCE:
29         return False # Pouce encore cart
30     return True
31
32
33 def detecter_geste(hand_landmarks):
34     """Classifie le geste de la main en une commande textuelle."""
35     # Poing : priorit maximale (arr t d'urgence)

```

```

36     if est_poing_strict(hand_landmarks):
37         return "stop"
38
39     d = etat_doigts(hand_landmarks)
40     nb_leves = sum(d)
41
42     if nb_leves >= SEUIL_PAUME_OUVERTE:          # 4 ou 5 doigts
43         avant
44         return "avant"
45     if d[1] and not d[2] and not d[3] and not d[4]:    # Index seul
46         gauche
47         return "gauche"
48     if d[1] and d[2] and not d[3] and not d[4]:        # Index+Majeur
49         droite
50         return "droite"
51
52     return None    # Geste ambigu : ignor
53
54 # Confirmation sur NB_FRAMES_CONFIRM frames consécutives
55
56 historique = []
57
58 def geste_confirme(nouveau_geste):
59     """Retourne le geste seulement s'il est stable sur plusieurs
60     frames."""
61     historique.append(nouveau_geste)
62     if len(historique) > NB_FRAMES_CONFIRM:
63         historique.pop(0)
64     if len(historique) == NB_FRAMES_CONFIRM and len(set(historique))
65     == 1:
66         return historique[0]
67     return None

```

Listing 4.4 – Fonctions de détection – vision.py

4.3.3 Boucle principale de capture et d'envoi

```

1 COULEURS = {
2     "stop" : (0, 0, 255),    # Rouge
3     "avant" : (0, 255, 0),   # Vert
4     "droite" : (255, 165, 0), # Orange
5     "gauche" : (255, 0, 255), # Magenta
6 }
7

```

```

8  cap = cv2.VideoCapture(0)      # Webcam index 0 (première caméra
    disponible)
9  commande_precedente = None
10
11 while cap.isOpened():
12     success, image = cap.read()
13     if not success:
14         break
15
16     # Conversion BGR      RGB pour MediaPipe
17     image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
18     resultats = hands.process(image_rgb)
19     image      = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)
20
21     geste_brut = geste_final = None
22
23     if resultats.multi_hand_landmarks:
24         for hand_landmarks in resultats.multi_hand_landmarks:
25             mp_drawing.draw_landmarks(image, hand_landmarks,
26                                       mp_hands.HAND_CONNECTIONS)
27             geste_brut = detecter_geste(hand_landmarks)
28
29     geste_final = geste_confirme(geste_brut)
30
31     # Envoi MQTT uniquement si geste nouveau et confirm
32     if geste_final and geste_final != commande_precedente:
33         client.publish(TOPIC, geste_final)
34         print(f"MQTT envoy : {geste_final}")
35         commande_precedente = geste_final
36
37     # Affichage en surimpression sur la vidéo
38     affichage = geste_final or geste_brut or "---"
39     couleur = COULEURS.get(affichage, (200, 200, 200))
40     cv2.putText(image, f"Geste : {affichage.upper()}",
41                 (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.2, couleur, 3)
42
43     # Barre de progression de confirmation
44     if geste_brut:
45         progression = int((len(historique) / NB_FRAMES_CONFIRM) *
46                             300)
47         cv2.rectangle(image, (10, 80), (10 + progression, 100),
48                       couleur, -1)
49         cv2.rectangle(image, (10, 80), (310, 100), (255, 255, 255),
50                       2)
51
52     cv2.imshow("Voiture Gesture Control", image)
53     if cv2.waitKey(5) & 0xFF == ord('q'):

```

```
51         break
52
53     cap.release()
54     cv2.destroyAllWindows()
55     client.loop_stop()
56     client.disconnect()
```

Listing 4.5 – Boucle principale – vision.py

Chapitre 5

Flux de Données et Protocoles

5.1 Le Protocole MQTT

MQTT (*Message Queuing Telemetry Transport*) est un protocole de messagerie *publish/subscribe* léger, idéal pour l'IoT. Son fonctionnement repose sur trois acteurs :

- Le **broker** (Mosquitto) : serveur central qui reçoit et redistribue les messages.
- Le **publisher** (`vision.py`) : publie une commande sur un *topic*.
- Le **subscriber** (`bridge.py`) : s'abonne au topic et réagit aux messages.

5.2 Communication Série (UART)

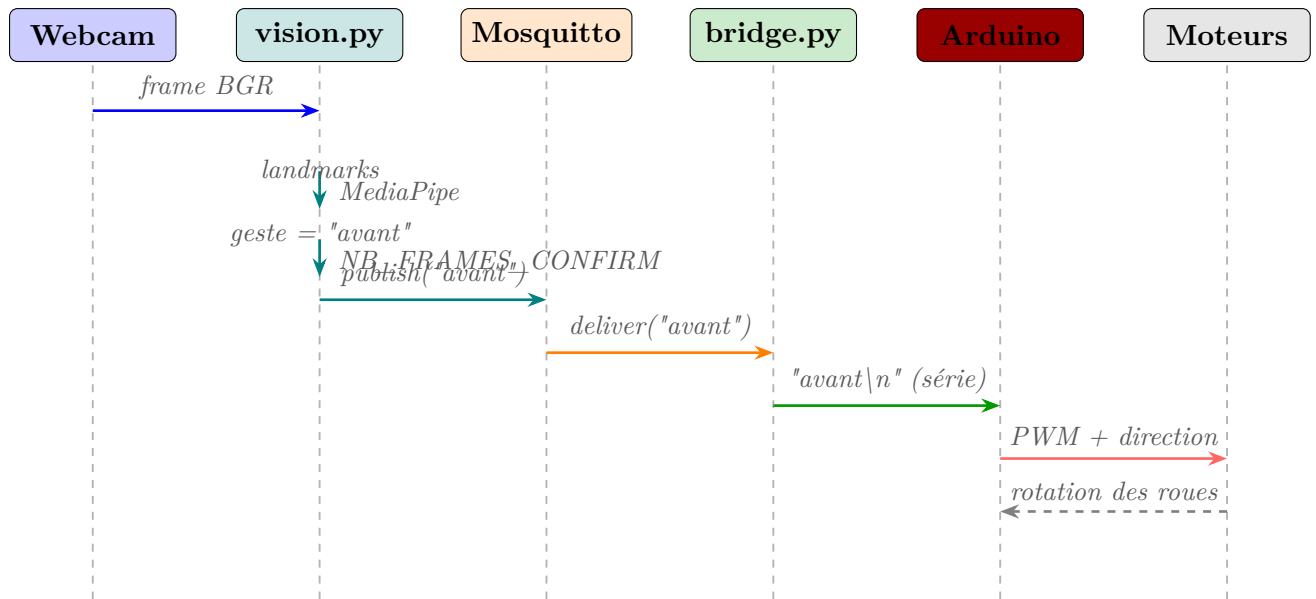
L'Arduino communique avec l'ordinateur via USB/UART à **9600 bauds**. `bridge.py` envoie une chaîne ASCII terminée par `\n`, que l'Arduino lit avec `Serial.readStringUntil('\n')`.

Trame :

a	v	a	n	t	\n
---	---	---	---	---	----

 (6 octets)

5.3 Diagramme de Séquence Complet



Chapitre 6

Mise en Œuvre et Démarrage

6.1 Ordre de Démarrage

Ordre impératif

Respectez impérativement cet ordre pour éviter les erreurs de connexion MQTT ou série.

1. **Téléverser le sketch Arduino** depuis l'Arduino IDE (une seule fois, ou après chaque modification du code Arduino).
2. **Démarrer le broker Mosquitto** (s'il n'est pas en service automatique) :

```
1 sudo systemctl start mosquitto
2 # V r i f i c a t i o n :
3 sudo systemctl status mosquitto
```

3. **Lancer bridge.py** (sur la machine connectée à l'Arduino) :

```
1 source env_voiture/bin/activate
2 python3 bridge.py
3 # Doit afficher : "En attente de commandes..."
```

4. **Lancer vision.py** (sur la machine avec webcam) :

```
1 source env_voiture/bin/activate
2 python3 vision.py
3 # U n e f e n e t r e v i d e o s ' o u v r e
```

5. **Présenter la main** devant la webcam et effectuer les gestes. Appuyer sur **q** pour quitter.

6.2 Résolution des Problèmes Courants

Symptôme	Cause probable	Solution
La webcam ne s'ouvre pas	Mauvais index de caméra	Essayez <code>cv2.VideoCapture(1)</code>
Gestes non détectés	Luminosité insuffisante ou fond complexe	Améliorer l'éclairage, utiliser fond uni
L'Arduino ne reçoit pas les commandes	Mauvais port série (<code>/dev/ttyACM?</code>)	Vérifier avec <code>ls /dev/ttyACM*</code> et <code>dmesg</code>
Mosquitto refusé (Connection refused)	Broker non démarré ou mauvaise IP	<code>sudo systemctl start mosquitto</code>
Moteurs tournent dans le mauvais sens	Polarité inversée sur L298N	Inverser les câbles du moteur concerné
La voiture ne bouge pas (moteur présent)	VITESSE trop faible, jumper EN-A/ENB actif	Augmenter VITESSE, retirer les jumpers ENA/ENB
Gestes confus / instables	Seuils trop permissifs	Augmenter <code>SEUIL_DOIGT_LEVE</code> et <code>NB_FRAMES_CONFIRM</code>

Chapitre 7

Pistes d'Amélioration

7.1 Améliorations Fonctionnelles

- **Marche arrière** : ajouter un geste (ex. pouce seul) et la fonction `reculer()` dans l'Arduino (inverser IN1/IN2 et IN3/IN4).
- **Vitesse variable** : encoder l'inclinaison du poignet (angle du vecteur base de la main) pour moduler le PWM proportionnellement.
- **Klaxon / LED** : associer des gestes secondaires à des sorties numériques (buzzer, LED RGB) sur l'Arduino.
- **Télécommande sans fil (Wi-Fi)** : remplacer la liaison série par un module ESP8266/ESP32 sur l'Arduino, supprimant le câble USB.
- **Deux mains** : utiliser `max_num_hands=2` dans `MediaPipe` pour des contrôles combinés (ex. main gauche = vitesse, main droite = direction).

7.2 Améliorations Techniques

- **Sécurité MQTT** : activer l'authentification Mosquitto (`password_file`) et TLS pour les déploiements hors réseau local.
- **Démarrage automatique** : créer des services `systemd` pour `bridge.py` et `vision.py` afin qu'ils démarrent automatiquement au boot du Raspberry Pi.
- **Journalisation** : remplacer les `print()` par le module `logging` avec rotation de fichiers pour faciliter le débogage.
- **Interface graphique** : ajouter un tableau de bord web (Flask + MQTT WebSocket) pour visualiser les gestes et l'état de la voiture à distance.

Chapitre 8

Conclusion

Ce projet illustre de façon concrète l'intégration de plusieurs technologies complémentaires : la **vision par ordinateur** avec OpenCV et MediaPipe, la **communication IoT** légère avec MQTT/Mosquitto, et la **programmation embarquée** sur Arduino pour le contrôle matériel.

L'architecture en trois modules découplés (détection, pont, actionneur) est délibérément modulaire : chaque composant peut être modifié indépendamment, ce qui facilite l'expérimentation et l'extension du projet.

Ce système constitue une excellente base pour explorer des domaines plus avancés : apprentissage automatique (classification de gestes par réseau de neurones), robotique mobile, interfaces homme-machine sans contact, ou encore la domotique pilotée par gestes.

Pour aller plus loin

- Documentation MediaPipe Hands : <https://mediapipe.readthedocs.io>
- Documentation Paho-MQTT : <https://eclipse.dev/paho/clients/python/docs/>
- Guide L298N + Arduino : <https://www.arduino.cc/en/Tutorial/LibraryExamples>
- Mosquitto MQTT Broker : <https://mosquitto.org/documentation/>